

Annotations & Reflection

Java Advanced Developer

StayAhead Training; Dec, 2025

About Annotations

Annotations & Reflection

- An annotation is a form of metadata, e.g. `@SomeMetadata`
- Annotations have no direct effect on your code
- They are used to:
 - help the compiler detect errors/suppress warnings
 - enable compile-time/runtime processing

Annotation Format

Annotations & Reflection

// without elements

@Override

// with one named element

@SuppressWarnings(value = "unchecked")

// with one unnamed element

@SuppressWarnings("unchecked")

// with two named elements

@LastModified(by = "Stuart", date = "15/12/2025")

This is a
made-up
annotation;
the others are
genuine Java
annotations

Annotation Format

Annotations & Reflection

// two or more on the same declaration

@Override

@SuppressWarnings("unchecked")

// with an element whose value is an array

@Target({ElementType.FIELD, ElementType.CONSTRUCTOR})

Applying Annotations

Annotations & Reflection

- **Declaration** annotation, e.g.:

```
// applicable to class, field, constructor, method etc.  
@Override  
public String toString() {...}
```

- **Type** annotation (since Java 8), e.g.:

```
// applicable to instantiation, cast, implements, & throws  
new @Interned Thing();
```

Declaring Annotations

Annotations & Reflection

```
public @interface LastModified {  
    String by();  
}
```

```
// applied to a class declaration  
@LastModified(by = "Stuart")  
public class SomeClass {...}
```

Annotation Retention

Annotations & Reflection

Retention policy	Description
SOURCE	The annotation is discarded by the compiler (exists in the <u>source code only</u>)
CLASS	The annotation is recorded in the class file but <u>not</u> retained at run time
RUNTIME	The annotation is recorded in the class file and retained at <u>run time</u> (available for <u>reflection</u>)

Declaring Annotations

Annotations & Reflection

```
@Retention(RetentionPolicy.RUNTIME)
```

```
public @interface LastModified {
```

```
    String by();
```

```
}
```

```
// applied to a class declaration
```

```
@LastModified(by = "Stuart")
```

```
public class SomeClass {...}
```

Annotation Target

Annotations & Reflection

Target (ElementType); to what can the annotation be applied?		
ANNOTATION_TYPE	CONSTRUCTOR	FIELD
LOCAL_VARIABLE	METHOD	PACKAGE
PARAMETER	TYPE (class/interface/enum)	TYPE_PARAMETER
TYPE_USE		

Declaring Annotations

Annotations & Reflection

```
@Retention(RetentionPolicy.RUNTIME)
```

```
@Target({ElementType.TYPE})
```

```
public @interface LastModified {
```

Note the braces - you may specify many ElementTypes

```
    String by();  
}
```

```
// applied to a class declaration
```

```
@LastModified(by = "Stuart")
```

```
public class SomeClass {...}
```

Declaring Annotations

Annotations & Reflection

- Each element is an abstract method with no parameters
- The element type may be primitive, `String`, `enum`, or an array of such
- Default values may be assigned via the `default` keyword
- The element named `value` may go unnamed

Declaring Annotations

Annotations & Reflection

```
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE})
public @interface LastModified {
```

```
    String[] by();
    String date();
    ChangeType type() default ChangeType.BUG_FIX;
    int linesChanged() default 1;
}
```

Built-in Annotations

Annotations & Reflection

Annotation	Elements	Description
@Deprecated	forRemoval since (version)	The element should no longer be used
@Override	N/A	The compiler should check the element overrides an inherited one
@SuppressWarnings	value (warnings)	The compiler should suppress the given warnings
@SafeVarargs	N/A	The dev. asserts the constructor/method does not perform unsafe ops. on the varargs param.
@FunctionalInterface	N/A	The type should be a functional interface

Built-in Annotations (for Annotations)

Annotations & Reflection

Annotation	Elements	Description
@Retention	value (ret. policy)	How the annotation is stored
@Target	value (element types)	To what can the annotation be applied
@Documented	N/A	The annotation should be included in the Javadocs
@Inherited	N/A	The annotation can be inherited
@Repeatable	value (anno. class)	The annotation can be applied more than once

About Reflection

Annotations & Reflection



Reflection is the inspecting, invoking, and modifying of code at runtime.

But why would I want to do that?
I have the source code!

Yes, but what if you want to build a framework.
It will have to work with code you didn't write.

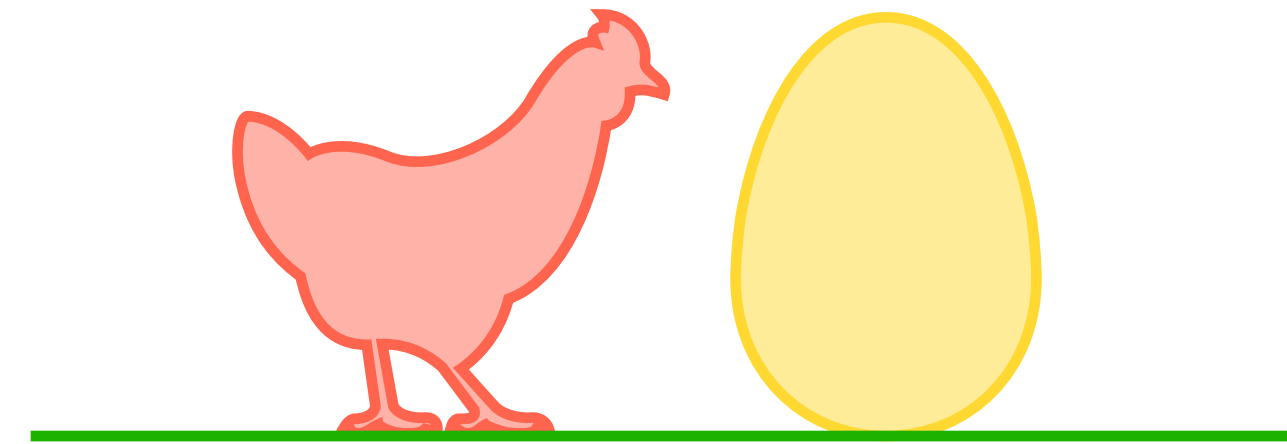
Ah, yes I see.



The Class Class

Annotations & Reflection

- Java classes are objects of type `Class`
- `Class` is a subclass of `Object`
- Get a reference to the `Class` object in one of two ways, e.g.:



```
Chicken chicken = new Chicken();
```

```
Class<? extends Chicken> cls1 = chicken.getClass();
```

```
Class<Chicken> cls2 = Chicken.class;
```

The Class Class

Annotations & Reflection

- Class objects may be loaded into memory by name, e.g.:

```
try {  
    Class<?> cls3 = Class.forName("com.example.ref.Chicken");  
    ...  
} catch (ClassNotFoundException e) {  
    // TODO  
}
```

Shallow Reflection

Annotations & Reflection

// get all public fields

```
Field[] fields = cls.getFields();
```

// get all public constructors

```
Constructor[] constructors = cls.getConstructors();
```

// get all public methods

```
Method[] methods = cls.getMethods();
```

// get all annotations

```
Annotation[] annotations = cls.getAnnotations();
```

Shallow Reflection

Annotations & Reflection

- To instantiate a class call newInstance on a Constructor instance, e.g.:

// assuming a no-args constructor

```
var instance cls.getConstructor().newInstance();
```

// assuming a constructor with one String parameter

```
var instance = cls.getConstructor(String.class)  
    .newInstance("Some value");
```

The argument

The parameter type

Shallow Reflection

Annotations & Reflection

- To invoke a method call `invoke` on a `Method` instance, e.g.:

```
// assuming a void method named hello with no params  
cls.getMethod("hello").invoke(instance);
```

```
// assuming an int method named cube with one int param.  
var result = cls.getMethod("cube", int.class)  
    .invoke(instance, 5);
```

The instance or
null for a
static method

The argument

The method name

The parameter type

Deep Reflection

Annotations & Reflection

- Access non-public members via `getDeclared[MemberType]` methods, e.g.:

```
Field privateNameField = cls.getDeclaredField("name");
```

- To make a field writable call `setAccessible` on the `Field` instance, e.g.:

```
privateNameField.setAccessible(true);
```

- To write to a field call `set` on the `Field` instance, e.g.:

```
privateNameField.set(instance, "New name");
```